Факультет компьютерных наук

Аналитика больших данных

Москва
2026

Валидация больших языковых моделей

План лекции

1. Что такое оценка LLM: уровни, цели, риски
2. Наборы задач и бенчмарки: дизайн, покрытие, утечки
3. Автоматические метрики для генерации
4. LLM-as-judge: протоколы, калибровка, смещения
5. Воспроизводимость и абляции
6. Диагностика галлюцинаций и вклад контекста/источников (RAG)

Оценивать LLM - зачем?

Твит Грега Брокмана президента компании OpenAI

Что значит оценивать LLM?

Оцениваем не только модель: промпт, данные, retrieval, инструменты, UX

Зачем оценивать

- Сравнить варианты (модель/промпт/RAG/инструменты)
- Найти регрессии до продакшена
- Понять почему стало хуже (диагностика)
- Оптимизировать trade-off: качество - стоимость - latency

Что может пойти не так

- Оптимизируем метрику, а не задачу
- Утечки и контаминация данных
- Смещения судьи (LLM-as-judge) и нестабильность
- Невоспроизводимые результаты (seed/версия модели/температура)

Таксономия оценок

Чтобы не смешивать точность, полезность и безопасность

	Reference-based	Reference-free
Автоматическая	BLEU/ROUGE/chrF BERTScore BLEURT/COMET	LLM-as-judge RAGAS/TruLens детекторы фактов
Человек	Аннотация качества pairwise предпочтения	UX/полезность выполнил задачу?

Уровни оценки

- Offline: статический набор задач/логов
- Online: A/B, метрики продукта, алерты
- Unit-тесты промптов/инструментов
- Safety: токсичность, утечки, jailbreak
- RAG: качество retrieval + groundedness

Наборы задач и бенчмарки

Типы наборов задач

- Закрытые ответы: классификация, multiple-choice (accuracy/F1)
- Открытые ответы: суммаризация, диалог, QA
- Код: unit tests, pass@k
- Инструменты/агенты: success rate, tool correctness

Типичные проблемы

- Контаминация (train/test leakage)
- Дрессировка на публичные бенчмарки
- Слишком узкое покрытие - слепые зоны
- Неясная спецификация (что считать правильным?)

Сделайте микробенчмарк из 50-200 примеров, отражающих ваши кейсы, и гоняйте его в CI на каждой итерации.

Как собирать набор задач (golden set)

Цель - покрытие, воспроизводимость и диагностичность

Хороший тест = делает ошибку видимой

Для диагностики храните ошибки как классы: фактичность, пропуски, неверные ссылки, формат, безопасность, tool-use.

Шаги

- Определить спецификацию ответа (rubric)
- Стратифицировать по подтипам ошибок/кейсам
- Добавить «хард» случаи (adversarial)
- Задать правила разметки + примеры
- Проверить согласие (inter-annotator)

Покрытие vs стоимость

- Глубина: больше примеров одного типа
- Ширина: больше типов запросов
- Сложность: multi-hop, длинный контекст
- Стабильность: фиксировать версии источников
- Сэмплинг: oversample редкие, но критичные баги

Автоматические метрики (классика)

Reference-based: overlap

- BLEU / chrF: машинный перевод
- ROUGE: суммаризация
- METEOR: леммы/синонимы
- Exact Match / F1: extractive QA
- Pass@k: код (с тестами)

Другие метрики (не только NLG)

- Классификация: accuracy, macro/micro F1, ROC-AUC
- Калибровка: ECE/Brier, reliability diagrams
- Ранжирование: NDCG/MRR/Recall@k
- Latency/cost: p50/p95, $/1k токенов
- Безопасность: токсичность/PII leakage/ASR на jailbreak

Семантические и обученные метрики

- Если есть референсы и важен смысл -> BERTScore/BLEURT (+ ROUGE для контроля).
- Если нет референсов -> переходите к LLM-as-judge/QA/NLI/контекстным метрикам.

Embedding-based

- BERTScore: сопоставление токенов по эмбеддингам
- Sentence-BERT cosine: быстро, но грубо
- MAUVE: сравнение распределений (divergence frontier)

Learned / model-based

- BLEURT: обученная метрика на синтетике + human judgements
- COMET: MT-метрика с учётом source + reference
- BARTScore / другие likelihood-метрики

Фактичность и галлюцинации: как измерять

Галлюцинация = утверждение, не поддержанное входом/источниками или реальностью

Автоматические подходы

- NLI/entailment: источник в ответ? (FactCC, SummaC)
- QA-based: задаём вопросы к ответу и источнику (QAGS)
- Claim extraction + retrieval + verify
- Citation check: поддерживает ли ссылка claim

Подводные камни

- NLI ломается на длинном контексте - нужно разбиение/агрегация
- QA/NLI чувствительны к шуму и формулировкам
- LLM-верификатор может согласиться с ошибкой
- Нужны негативные примеры и калибровка порогов

Полезная декомпозиция

Отдельно меряйте: (1) релевантность ответа, (2) groundedness/faithfulness, (3) качество retrieval (покрытие и точность контекста).

Оценка сгенерированных тестов/заданий

Когда LLM генерирует сам примеры для eval-а

Что валидируем

- Корректность (есть ли правильный ответ?)
- Однозначность формулировки
- Сложность/дискриминативность
- Диверсификация (без дублей/парафраз)
- Отсутствие утечек (секреты/PII/копипаст)

Авто-фильтры

- NLI: проверка референс в утверждение/ответ
- BERTScore: семантическая близость к эталону
- Комбинации: NLI (фактичность) × BERTScore/ROUGE (смысл/формулировка)
- Фреймворки: OpenAI Evals, lm-eval-harness, HF Evaluate, DeepEval/TruLens/RAGAS для пайплайнов

Генерация кандидатов → Дедуп/кластер → Авто-валидация (NLI/BERTScore) → LLM-ревью (рубрика) → Human spot-check → В выпуск

LLM-as-judge

Судья как дешевый человек: удобно, но не без подвохов

Протоколы

- Скалярный скоринг по рубрике (0–5)
- Pairwise: A vs B (win-rate)
- Multi-judge: ансамбль судей
- Сэмплинг 3–5 прогонов для оценки дисперсии

Best practices

- Фиксируйте rubric + примеры идеал/провал
- Рандомизируйте порядок ответов (A/B), скрывайте модель
- Контролируйте длину (verbosity bias)
- Калибруйте на человеческой подвыборке

Известные смещения

Position bias, verbosity bias, self-preference/self-enhancement, sensitivity к формату промпта. Используйте рандомизацию и контрольные тесты.

Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Фреймворки для оценки и валидации

Бенчмарки / harness

- lm-evaluation-harness (EleutherAI)
- HELM (Stanford CRFM)
- OpenAI Evals / Evals API
- Hugging Face Evaluate (метрики)

LLM apps / RAG

- RAGAS: faithfulness, answer/context relevancy
- TruLens: RAG triad + трассировка
- LangChain OpenEvals / LangSmith (если используете экосистему)
- Regression тесты промптов (например promptfoo и аналоги)

- Если у вас много задач/датасетов - harness (lm-eval/HELM).
- Если у вас приложение - то трассировка + метрики для компонентов (RAGAS/TruLens + собственные проверки).

Воспроизводимость: чек-лист

Чтобы результаты можно было повторить через неделю/месяц

Фиксируйте ✓

- Версию модели (ID) и параметры декодинга
- Промпты/шаблоны и system prompt
- Версии данных/источников (snapshot)
- Seed + число прогонов (n) для нестабильности
- Окружение: код, зависимости, конфиг

Логируйте ✓

- Вход, контекст (retrieved docs), выход
- Токены/стоимость/latency
- Trace шагов агента и tool calls
- Версии индекса/эмбеддингов/ранкера в RAG
- Ошибки и исключения (с категориями)

Сводка метрик + распределения, список top regressions и ссылки на логи для реплея.

Абляции: понять вклад компонентов

Ablation ≠ поменяли всё - меняем один фактор за раз (или факторный дизайн)

Примеры абляций для LLM приложений

- Температура/decoding (greedy vs sampling)
- Промпт: убрать один блок инструкций
- RAG: без retrieval / другой chunking / другой reranker
- Инструменты: выключить один tool
- Пост-обработка: выключить правила/ валидаторы

Как делать правильно

- Одинаковые входы, одинаковые условия
- Достаточно прогонов (дисперсия)
- Стратификация по типам задач
- Факторный план для взаимодействий (prompt × retrieval)
- Отчёт: эффект + доверительные интервалы

Вклад контекста и источников

Если модель галлюцинирует - виноваты ли retrieval, контекст или генерация?

Тесты на вклад контекста

- Context ablation: убрать retrieved docs и сравнить
- Leave-one-source-out: по одному исключать источники
- Shuffle/replace: подменить контекст нерелевантным
- Контроль длины: truncation vs full context

Тесты на groundedness

- Claim-level проверка: каждый claim должен иметь evidence
- Citation precision/recall (ссылки - это evidence)
- NLI/QA верификация по источникам
- LLM-judge с обязательным цитированием

Если без контекста качество не падает - retrieval бесполезен. Если падает релевантность, но растёт фактичность - контекст шумный/ложный.

RAG: что измерять по компонентам

Разделяем retrieval и generation - иначе не диагностируем

Компонент	Что меряем	Примеры метрик
Retrieval	Насколько контекст покрывает вопрос и не содержит мусора	Recall@k, precision@k, context relevance
Groundedness	Поддерживаются ли утверждения evidence из контекста	Faithfulness (RAGAS), NLI/QA verify
Answer quality	Полезность/полнота/структура ответа	LLM-as-judge rubric, win-rate, task success

Если есть разметка evidence - отлично. Если нет, используйте LLM-оценку groundedness, но калибруйте и проверяйте на негативных примерах.

Практический eval loop (как в продакшене)

CI + мониторинг + быстрый дебаг по логам

Spec (рубрика) → Данные (golden + логи) → Раннер (harness) → Метрики + судья → Отчёт + алерты → Итерация (абляции)

Гейтинг релиза

- Regression budget: сколько можно упасть
- Список must-not-fail кейсов
- Разные пороги по сегментам (важные пользователи)

Мониторинг в онлайне

- Логи + трейсинг + выборочная разметка
- Алерты на всплеск галлюцинаций/ошибок
- Canary/A-B: проверка на реальном трафике

Итоги

- Начинайте со спецификации качества (rubric) и целевой метрики продукта.
- Делайте гибрид: быстрые метрики -> скрининг, LLM-as-judge -> масштаб, человек -> калибровка.
- Для фактичности нужны контекстные проверки (NLI/QA/claim+evidence), а не только ROUGE/BLEU.
- Воспроизводимость - это логи + версии + параметры; без неё сравнение бессмысленно.
- Диагностика требует абляций и разложения по компонентам (retrieval vs generation)

ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •